

面向 MoE 推理 E/A 分离场景的异构算力协同调度系统设计

AFD Demo Project

0.1. 摘要

本项目面向大规模 MoE 模型推理中的 Attention/Expert 分离场景，构建了一套异构算力协同调度与通信时延掩盖系统。系统将 Transformer 层中的 Attention 计算、KV cache、logits 与采样职责放在 Attention 侧，将 post-attention norm、MoE router、专家计算与 combine 职责放在 FFN/Expert 侧，并通过 Dual Batch Overlap (DBO)、cross-layer pipeline、EP overlap 和可插拔通信后端减少前后端协同时延。

设计重点包括三点。第一，解耦控制链路和数据链路：控制面低频维护 worker 注册、路由表、expert ownership 和负载指标，数据面由 Attention 与 FFN/Expert rank 直接执行 P2P 或 EP group 通信。第二，面向微秒级计算阶段进行流水线化：microbatch、decode cross-layer、early receive 和异步 EP work item 共同扩大计算覆盖通信的窗口。第三，针对 Ascend 910C/HCCCL 环境提供可验证、可回退的通信体系：当前稳定基线为 broadcast_reduce_overlap, all_to_all_single、sparse_p2p_overlap、official MoE v2 与 DeepEP 作为分阶段演进路线。

当前系统已经具备真实 Qwen3-30B-A3B 路径、NPU 910C 跨机实验脚本、timing JSON、pipeline 图、msprof 分析和通信架构对比报告。现阶段结论是：E/A 分离和 EP 扩展能显著缩短单层 FFN local expert compute，但端到端 decode DBO 是否超过 serial 仍取决于通信固定开销、HCCCL collective 形态和调度气泡能否被计算充分掩盖。

目录

0.1. 摘要	1
1. 背景与挑战	1
2. 总体架构	2
3. 去中心化 E/A 分离协同调度	2
3.1. 控制链路和数据链路解耦	2
3.2. 静态路径与 Coordinator 路径并存	3
4. 通信时延掩盖设计	3
4.1. Dual Batch Overlap	3
4.2. Decode cross-layer pipeline	3
4.3. EP overlap work item	3
5. MoE EP 通信后端	4
5.1. broadcast_reduce_overlap	4
5.2. all_to_all_single	4
5.3. sparse_p2p_overlap	4
5.4. Official MoE v2 与 DeepEP	4
6. 技术先进性	4
7. 可行性分析	5
7.1. 已落地工程路径	5
7.2. 已验证结果与当前限制	5
8. 性能评估方法	5
9. 风险、限制与演进路线	6
9.1. 风险与限制	6
9.2. 演进路线	6
10. 结论	6

1. 背景与挑战

MoE 模型推理具有明显的异构执行特征：Attention 侧主要受 KV cache、Q/K/V 投影、attention kernel 与 logits 计算影响；FFN/Expert 侧主要受 router、top-k 专家激活、expert matmul、dispatch/combine 通信影

响。对于 Qwen3-30B-A3B 这类稀疏激活模型，每个 token 只激活部分专家，但模型仍包含大量专家权重和复杂的路由逻辑。

在 E/A 分离部署下，系统希望把 Attention 与 Expert 计算放在不同设备、不同 rank 或不同主机上，从而利用异构资源和 expert parallelism。然而，这会引入新的协同开销：

- Attention 与 FFN/Expert 之间每层都需要传输 hidden states。
- FFN EP rank 之间需要 dispatch routed tokens，并将 expert outputs combine 回 coordinator。
- Attention 和 FFN 的单层计算时间都可能处于微秒到毫秒级，通信固定延迟已经不可忽略。
- 如果控制链路和数据链路耦合，路由表查询、worker 状态同步和数据发送会互相阻塞。
- EP 规模扩大后，local expert compute 下降，但 collective 次数、变长 split、metadata exchange 和同步等待可能抵消收益。

因此，本项目的核心目标不是简单将模型切开，而是在 E/A 分离后同时解决“谁计算”“谁通信”“何时通信”“如何观测”的问题。

2. 总体架构

系统的真实推理入口是 `src/main.py`。它完成设备后端初始化、分布式拓扑构建、模型加载、scheduler 选择、timing 输出和可选 coordinator routing。核心模型位于 `src/model/`，流水调度位于 `src/pipeline/`，rank/group 抽象位于 `src/distributed/`，NPU/CUDA/CPU 设备抽象位于 `src/utils/device.py`，新一代控制面与可插拔通信接口位于 `src/coordinator_arch/`。

整体数据流如下：

```
input token ids
-> Attention role: embedding, self-attention, KV cache, lm_head, sampling
-> A2F hidden states
-> FFN coordinator: residual merge, post-attention norm, router/gate
-> FFN EP ranks: local experts
-> FFN coordinator: combine/reduce, residual output
-> F2A hidden states
-> next Attention layer or final logits
```

系统中的主要角色包括：

- Attention rank: 持有 embedding、self-attention、KV cache、final norm、lm head 与 sampling。KV cache 的所有权保留在 Attention 侧，FFN/Expert 侧不持有 KV cache。
- FFN coordinator: 对接 Attention rank，负责接收 A2F hidden states、执行 post-attention norm 与 router、发起 EP dispatch、收集 expert outputs，并通过 F2A 返回 hidden states。
- FFN expert ranks: 只处理本 rank 拥有的 local experts。它们不直接与 Attention rank 通信，只参与 FFN EP group 内部 collective 或 P2P。
- Coordinator 控制面: 维护 worker registry、routing table、expert ownership 与 metrics。它补充静态 A/F 路径，不强制替换当前生产/验证路径。

Serial baseline 在本项目中仍然是 A/F 分离路径，只是关闭 DBO。它不是 monolithic model baseline。因此 decode speedup 必须使用同拓扑 serial 的 `decode_tpot_ms` 作为 denominator。

3. 去中心化 E/A 分离协同调度

3.1. 控制链路和数据链路解耦

控制面用于低频决策，数据面用于高频 token hidden states 传输。二者解耦后，Attention 与 FFN/Expert 之间的数据不需要经由中心控制器中转；控制器只发布路由表、expert ownership 和负载信息。

控制链路承担：

- worker 注册与心跳；
- routing table 版本管理；
- expert 到 rank 的 ownership 映射；
- worker metrics 与未来 EPLB 负载均衡输入；

- safe-point routing update。

数据链路承担：

- A2F/F2A hidden states P2P;
- FFN EP 内部 dispatch/combine;
- broadcast_reduce_overlap、all_to_all_single、DeepEP 或 official MoE op 等可插拔 communicator。

这种设计避免在每个 token 或每层都向控制器查询路由，降低控制面抖动对数据面延迟的影响。同时，路由表作为版本化本地缓存存在于 worker 侧，后续可在 layer/step safe point 切换，降低动态负载均衡带来的协议风险。

3.2. 静态路径与 Coordinator 路径并存

当前系统保留两条路径：

- 静态 A/F + EP path: DisaggregatedQwenModel、AttentionWorker、FFNWorker、EPFFNLayer 和 decode scheduler 组成当前主要性能验证路径。
- Coordinator path: src/coordinator_arch/ 提供 gRPC 控制面、routing table、worker skeleton 与 MoECommunicator 抽象，用于动态专家放置、EPLB 和未来 DeepEP/official MoE 后端。

并存的原因是工程可行性。静态路径已经能加载真实 Qwen3 权重并在 Ascend 910C 上跑端到端实验；Coordinator 路径更适合引入动态路由和专家副本，但仍需要逐步把 skeleton worker 与真实 FFN compute 对齐。

4. 通信时延掩盖设计

4.1. Dual Batch Overlap

DBO 将 batch 沿 batch 维拆分为多个 microbatch，使 Attention 与 FFN 在层间形成流水。对于 prefill，AsyncPipelineScheduler 使用 MicroBatchManager 切分 batch；对于 decode，DecodeDBOScheduler 根据 --num-micro-batches 计算 microbatch size，并对 input ids、position ids、attention mask、KV cache slice 和 A/F hidden states 使用同一切分。

典型目标是把当前 microbatch 的 FFN/通信与下一个 microbatch 的 Attention 计算重叠：

```
MB0: Attention(L) -> A2F -> FFN(L) -> F2A
MB1:                Attention(L) -> A2F -> FFN(L) -> F2A
```

microbatch 数量不是单层 Attention kernel 优化，它改变的是端到端 pipeline 结构。当前实验显示，MB3/MB4 能运行，但并不必然优于 MB2；过小 microbatch 可能导致 expert GEMM 更碎、launch overhead 上升。

4.2. Decode cross-layer pipeline

Decode cross-layer 进一步允许不同 layer、不同 microbatch 交错，使 Attention 尽早进入下一层，减少整层等待。它的关键约束是所有 FFN EP ranks 必须按相同 layer-major、microbatch-major 顺序进入 HCCL collective，否则会出现死锁或 payload/tag 错配。

最新 early receive 调度优化的核心思想是：当某个 (layer, microbatch) 的 F2A 已发出后，如果下一层存在，就尽早 post 下一层同一 microbatch 的 A2F receive。这样 Attention 侧可以更早发送下一层 MB0，而不是等 FFN 侧处理完整层所有 microbatch 后才匹配 receive。

4.3. EP overlap work item

EPFFNLayer 将 FFN EP 的单个 microbatch 拆成 work item：

```
create_work_item -> dispatch_async -> finish_dispatch
                  -> compute_local -> reduce_async/combine_async
                  -> finish_reduce/finish_combine -> finish_output
```

在 broadcast_reduce_overlap 中，通信语义仍是 full hidden/router broadcast + dense partial reduce，但调度顺序允许上一 MB 的 reduce 与下一 MB 的 local expert compute 重叠。这个路径没有减少字节数，收益来自更好的时间安排。

5. MoE EP 通信后端

5.1. broadcast_reduce_overlap

这是当前真实 decode EP 默认推荐路径。FFN coordinator 将完整 hidden_2d、selected_experts、routing_weights broadcast 给所有 EP ranks；每个 rank 只计算本地专家，输出 dense partial [tokens, hidden]；最后 reduce/sum 回 coordinator。

优点：

- HCCL collective 形态简单；
- 所有 rank 的顺序易于控制；
- 真实 Qwen3 路径稳定；
- 当前实测快于通用 all-to-all fallback。

缺点：

- 没有充分利用 MoE token/expert 稀疏性；
- dispatch/reduce payload 随 EP 和 hidden size 增长；
- 当 local FFN 已与 Attention 对齐后，通信和 recv wait 会成为主要瓶颈。

5.2. all_to_all_single

该路径将 routing 展开为 (token, expert) assignment，根据 expert ownership 发送到目标 rank，再将 assignment outputs gather 回 coordinator 并按 top-k weight combine。它理论上更稀疏，避免 full hidden broadcast 和 dense reduce。

当前实现已接入真实 EPFFNLayer，但不推荐作为默认路径。NPU msprof 显示它落到 HCCL hcom_alltoallv_，并引入 count exchange、metadata broadcast、变长 split、排序/恢复等开销。b32/s256 下逻辑 payload 很小，但 dispatch 耗时显著高于 broadcast/reduce，说明瓶颈不是物理带宽打满，而是 collective 固定开销和协议开销。

5.3. sparse_p2p_overlap

该路径的理论动机是：真实 source 只有 FFN coordinator，不一定需要 EP 全体 all-to-all。Coordinator 可以按目标 EP rank 直接发送 packed assignments，expert rank 计算后返回 packed outputs。

CPU/Gloo reference 已经能验证 assignment combine 语义，但 Host1 NPU EP7 smoke 曾出现非本地 expert id 和异常 count，说明 HCCL NPU P2P 在多 peer、多 payload、同源 coordinator 模式下仍存在 tag/payload 匹配或使用方式风险。因此它当前保留为实验和最小复现入口，不进入性能矩阵。

5.4. Official MoE v2 与 DeepEP

torch_npu.npu_moe_distribute_dispatch_v2/combine_v2 与 DeepEP 更接近长期目标形态。它们理论上能把 dispatch/combine、layout、count 和通信策略交给更贴近硬件的实现。

当前状态仍是 experimental：

- torch_npu 2.6 环境下，official base/v2 对 Qwen3 hidden=2048 或 comm_alg 暴露存在限制。
- 隔离 torch_npu 2.9/CANN 8.5.1 容器中，dispatch_v2 对 H=2048 与 comm_alg=fullmesh_v2 有进展。
- 真实 Qwen nonzero expert output 曾在 combine_v2 触发 aicore timeout；zero-experts 或 synthetic 路径不能代表生产可用。
- DeepEP 需要继续在 normal/low_latency RT bench 与真实端到端路径中验证。

6. 技术先进性

本项目的技术先进性体现在以下方面。

第一，面向 MoE 推理的角色级解耦。系统不是只优化单个 kernel，而是从模型结构层面将 Attention/KV cache 与 Expert/FFN 计算分离，使两类计算可以独立扩展、独立放置和独立 profiling。

第二，控制面与数据面解耦。Coordinator 只承担低频路由控制和负载状态同步，数据面由 worker 直接建立 P2P、collective 或 communicator 通信。这种设计避免中心控制器成为数据热路径瓶颈，也为专家副本和动态负载均衡提供基础。

第三，面向通信掩盖的多级流水。DBO、decode cross-layer、EP overlap、early receive 和 async communicator 共同作用，使通信不只是被测量，而是被主动调度到可被计算覆盖的位置。

第四，可插拔通信后端。broadcast_reduce_overlap、all_to_all_single、sparse_p2p_overlap、DeepEP、official MoE op 共享 EPFFN/communicator 抽象，允许在相同模型和相同 timing 口径下做 apples-to-apples 对比。

第五，NPU/HCCL 实证驱动。系统内置 910C 运行脚本、跨机 fresh port 配方、msprof 采集、timing JSON、pipeline 图和 bandwidth lower-bound 分析，能把“理论稀疏通信是否更快”落实到真实硬件数据。

7. 可行性分析

7.1. 已落地工程路径

当前仓库已经具备：

- src/main.py: 统一 CLI、分布式初始化、scheduler 选择与 timing 输出。
- src/model/disaggregated.py: 真实 Qwen3 A/F 分离模型路径。
- src/model/attention_worker.py: Attention role、KV cache、official NPU Attention kernel、RMSNorm/RoPE fusion、layer input precopy。
- src/model/ffn_worker.py: FFN role 和非 EP FFN layer。
- src/model/ep_moe.py: EP expert sharding、broadcast/reduce overlap、all-to-all backend、sparse P2P experimental backend。
- src/pipeline/decode_scheduler.py: decode DBO、cross-layer、EP overlap decode path。
- src/coordinator_arch/: Coordinator 控制面、routing table、fallback communicator 和 worker skeleton。
- scripts/run_npu.sh、scripts/run_crosshost_static_ep_*: NPU 单机与跨机实验入口。

这些路径说明系统不是概念设计，而是可以加载真实模型权重、生成 timing、画 pipeline 图并在 Ascend 910C 上验证的工程实现。

7.2. 已验证结果与当前限制

已有实验表明，扩大 FFN EP 能显著降低 local expert compute。在跨机 Host1 Attention + Host2 EP16 的 b32/s256/t20 配置下，EP16 相比 EP8 明显降低 FFN local experts 与单层 FFN 时间；b256/s1024 下 local FFN 已接近 Attention。

但端到端 decode DBO 仍可能低于同拓扑 serial。根因不是 FFN local compute 未优化，而是 dispatch/reduce、A2F/F2A 等通信与 Attention recv wait 仍无法完全被计算掩盖。all_to_all_single 虽有稀疏语义，但当前 HCCL all-to-all 固定开销高，实测慢于 broadcast/reduce。

因此系统当前的可行路线是“稳定路径 + 实验路径”并行：保留 broadcast_reduce_overlap 作为默认可比较基线，在隔离环境继续推进 padded/equal-split all-to-all、official MoE v2 和 DeepEP。

8. 性能评估方法

系统采用以下指标评估优化是否有效：

- prefill_ms: prefill 阶段耗时。
- decode_tpot_ms: decode 阶段 batch-level 每步耗时，是 decode speedup 的主指标。
- speedup = serial / DBO: 大于 1.0x 才表示 DBO 更快。
- attention_avg_layer_ms_excl_l0: 排除第 0 层后的 Attention 平均单层时间。
- ffn_avg_layer_ms_excl_l0: 排除第 0 层后的 FFN 平均单层时间。
- ep_dispatch_avg_layer_ms_excl_l0、ep_reduce_avg_layer_ms_excl_l0: EP 通信阶段耗时。
- attention_recv_wait_avg_layer_ms_excl_l0: Attention 等待 FFN 返回的气泡。
- effective_bandwidth: 用逻辑 payload 下界除以 timing 耗时估算，不等同于物理链路峰值。
- NPU 利用率、HBM、msprof Communication Time、HCCL op summary: 用于定位瓶颈。

代表性实验命令包括：

```
./scripts/run_npu.sh --attn-size 1 --ffn-size 1 --ffn-tp-size 1 \
--batch 8 --seq 128 --tokens 20 --model-name "$MODEL_NAME"
```

```
python scripts/run_crosshost_static_ep_matrix.py \
--ep-sizes 16 \
--backends broadcast_reduce_overlap,all_to_all_single \
--modes serial,decode-dbo,decode-dbo-crosslayer \
--configs 32:256 \
--tokens 20 \
--attn-kernel npu-official \
--attn-precoppy-layer-inputs \
--attn-fused-rmsnorm \
--attn-fused-rope
```

报告与图形由 `scripts/gen_experiment_report.py`、`scripts/visualize_dbo_pipeline.py`、`scripts/plot_all_pipelines.py`、`scripts/summarize_crosshost_ep_timing.py` 和 `msprof` 后处理脚本生成。

9. 风险、限制与演进路线

9.1. 风险与限制

- HCCL collective 顺序必须严格一致。所有 FFN EP ranks 都要以相同 layer-major、microbatch-major 顺序进入 dispatch、local compute、reduce/combine。
- Decode pipeline 图只展示 0-based decode step 1，用于观察 overlap，不作为最终 speedup denominator。
- all_to_all_single 的逻辑 payload 更小，但 HCCL alltoallv 固定开销和 metadata/count exchange 当前较高。
- NPU sparse P2P backend 在真实 HCCL 多 peer、多 payload 场景中仍 blocked。
- official MoE v2 在 synthetic 或 zero-experts 中通过不代表真实 Qwen nonzero expert output 稳定。
- Host2 磁盘与 msprof raw profile 空间可能限制全 rank profiling。

9.2. 演进路线

短期：

- 保持 broadcast_reduce_overlap 作为真实 decode EP 默认 baseline。
- 继续优化 early receive、cross-layer 调度和 timing 可观测性。
- 对代表配置做 targeted msprof，而不是全矩阵 profile。

中期：

- 原型化 padded/equal-split all-to-all，减少变长 alltoallv 固定开销。
- 合并 metadata collective，减少每 layer/MB 的 count/expert metadata exchange。
- 给 FallbackMoECommunicator 增加更细 timing，拆分 count exchange、hidden all-to-all、metadata broadcast、combine restore。

长期：

- 在隔离容器继续推进 official MoE v2 nonzero expert combine 稳定性。
- 修复 DeepEP normal/low_latency 最小 RT bench 后接入真实 EPFFN。
- 结合 reserved NPU pool、routing table 和 worker metrics 实现 EPLB 热专家副本。

10. 结论

本项目已经形成一套可运行、可观测、可迭代的 E/A 分离 MoE 推理系统。其核心价值在于把 Attention 与 Expert 侧的职责、控制面与数据面、计算与通信调度、稳定路径与实验路径清晰解耦，并在 Ascend 910C/HCCL 环境中用真实模型和真实通信验证设计取舍。

当前阶段最务实的选择是：以 broadcast_reduce_overlap 作为稳定基线，继续通过调度优化扩大通信掩盖窗口；同时把 all_to_all_single、sparse P2P、official MoE v2 和 DeepEP 作为有明确验证门槛的演进

方向。只有当真实 decode TPOT、数值正确性、msprof 通信分析和跨机稳定性同时证明收益后，实验通信后端才应进入默认路径。